

AI Engineer Take-Home

Market Signal Intelligence Bot

Target level: Intermediate to Senior | Suggested time: 4-8 hours

Build a small system that reads noisy group-chat messages, extracts useful market/resource signals, and turns them into a compact view of resource trends over time.

Context

Imagine several Telegram and WeChat groups where people discuss supply and demand for AI resources (for example model/API access, credits, accounts, capacity, quotas, or related inventory). Messages are informal, incomplete, duplicated, conversational, and sometimes contradictory. We want to turn this stream into something operationally useful.

Your task

Design and implement as much of the pipeline as you reasonably can within 4-8 hours. The goal is not to finish a production system. We care more about how you define the problem, choose abstractions, handle ambiguity, and make tradeoffs than about feature count.

Scope intentionally remains open. You may simplify integrations, use sample/replayed messages, mock WeChat ingestion, focus on Telegram, or build a local prototype. Clearly state what you implemented, what you mocked, and what you would do next.

Minimum outcome

Given a stream or dataset of chat messages, produce a structured representation of useful resource signals and at least one aggregated trend view. The trend can be a dashboard, chart, table, API response, notebook, or another format you believe communicates the market state well.

Examples of the kind of input you may encounter

```
"Claude 4.2 available, around 100k."  
"Still looking for 500k OpenAI, below 4 if possible."  
"The 3.8 seller from yesterday is gone."  
"AWS seems harder to get this week."  
A: "Does the one from earlier still have stock?"   B: "Yes."   A: "I can take 200 at 3.8."
```

These examples are illustrative only. You are encouraged to define a better schema, generate additional test cases, or identify information that should *not* be treated as a reliable signal.

What we would like to see

There is no required architecture. The prompts below are guidance, not a checklist. Strong submissions may go deep on only a subset if the choices are well reasoned.

Area	Questions worth thinking about
Signal definition	What is a signal? What fields matter? Can one message contain multiple signals? How do you distinguish supply, demand, trans
Conversation context	When is a single message insufficient? How would replies, quotes, prior messages, or conversational state affect extraction?
Normalization	How do aliases and related resources map to canonical entities? How would you handle unknown/new resource types without r
Quality & confidence	How do you represent uncertainty? What happens when messages conflict, quantities are missing, or the model is unsure?
Duplication & provenance	If five people repeat the same quote, is that five market observations or one underlying event? How would you preserve eviden
Aggregation	What does a useful trend mean? Consider price, quantity/volume, availability, sample count, dispersion/variance, or other metri
LLM/system design	Where should an LLM be used, and where should deterministic logic be used? How would your design behave at 10x or 100x m
Evaluation	How would you know the system is working? What would you label or measure? What failures matter most to the end product?

A possible output - not a required one

For each canonical resource, you might expose a time series with a central price estimate, observed range or dispersion, available volume, number of independent signals, and confidence. A strong submission should be able to explain why its aggregation is meaningful rather than simply plotting an average.

Example: Claude / AWS median price 3.8 | observed volume 1.3M | dispersion rising | 12 independent signals | confidence 0.82

Data & integration expectations

Do not spend the entire exercise fighting third-party platform access. Real Telegram ingestion is welcome. For WeChat, a mock adapter, exported/replayed messages, or a clearly described interface is acceptable if direct integration would consume disproportionate time.

We recommend creating a small but adversarial dataset.

Include clean messages as well as difficult cases: follow-up replies, missing units, aliases, repeated offers, forwarded information, contradictory quotes, historical references, irrelevant chatter, and messages whose meaning depends on prior context. You may synthesize these examples.

Suggested interface boundary

A useful design may separate platform ingestion from intelligence. For example, both Telegram and WeChat could emit a common message/event format into the same downstream pipeline. This is only a suggestion - choose a different boundary if you can justify it.

Constraints

Timebox: 4-8 hours. Please stop when the timebox is reached and tell us what remains.

Models/tools: Use any LLM, framework, database, visualization library, or coding assistant you prefer.

Language: Any reasonable implementation language is acceptable.

External services: Do not include private credentials. Mock paid or inaccessible services when appropriate.

AI assistance: Allowed. We care about the resulting engineering judgment and your ability to explain it.

What to submit

1	Code - a runnable repository or compact project.
2	README - setup/run instructions and a short architecture explanation.
3	Demo output - screenshot, dashboard, chart, sample API output, or generated report.
4	Design notes - assumptions, tradeoffs, known failure modes, and what you would build next.
5	Evaluation - even a small hand-labeled set or a proposed evaluation plan is valuable.

Please include a short note stating approximately how long you spent.

Optional extensions

If you have time and want to demonstrate depth, choose whatever extension best shows your strengths. Examples include streaming ingestion, entity ontology, signal provenance/deduplication, confidence calibration, incremental aggregation, anomaly detection, multilingual extraction, cost-aware model routing, evaluation tooling, or a polished interactive dashboard. None is required.

How we evaluate the submission

We are hiring for an intermediate-to-senior AI Engineer. We do not score primarily by the number of completed features. We look for evidence that you can take an ambiguous AI problem and turn it into a robust system.

Dimension	What strong work tends to demonstrate	Weight
Problem framing	Clear definition of signal, useful abstractions, sensible assumptions, awareness of ambiguity.	20%
AI / extraction design	Good model boundaries, context handling, structured extraction, uncertainty and failure handling.	20%
System engineering	Clean interfaces, data flow, storage choices, reliability, extensibility, observability or scaling judgment.	20%
Data reasoning	Thoughtful normalization, dedup/provenance, aggregation, statistics, and interpretation of trends.	15%
Evaluation	A credible way to measure extraction and/or end-product quality; useful adversarial cases.	15%
Communication & prioritization	Readable code/docs, explicit tradeoffs, good use of the 4-8 hour timebox.	10%

What we are not looking for

A giant amount of boilerplate, a polished UI hiding a weak data model, blindly sending every message to one LLM without considering context/cost/reliability, or an over-engineered architecture that cannot be demonstrated within the timebox.

Discussion after the exercise

Be prepared to walk us through one or two difficult examples, explain a major design tradeoff, identify where your system will fail, and describe how you would evolve it toward production. We may also change one assumption during the discussion and ask how your design adapts.

The strongest submissions usually make us want to ask: “Why did you choose this?” - and have a thoughtful answer.

Notes for candidates

We intentionally leave parts of the problem underspecified. Asking good questions is valuable, but you do not need to wait for clarification: make reasonable assumptions, document them, and move. We encourage you to use the exercise to show the kind of engineering work you are strongest at.